

Experiment 11 — Fourth sweep: closing the distance to NumPy

Mathilda — execution-speed experiments

Contents

Experiment 11 — Fourth sweep: closing the distance to NumPy	1
Hypothesis	1
The probe (10^6 float64)	1
Six fixes	2
Results	4
A separate finding: dgemm slows the next threaded loop by 1.45×	4
What the tests missed — three times	4
Verification	5
Still open	5
Why Mathilda is not the fastest here, and what it would take	5

Experiment 11 — Fourth sweep: closing the distance to NumPy

Date: 2026-07-31 · Code: `src/ndstruct.{c,h}`, `src/ndreduce.{c,h}`, `src/convolutions.c`, `src/numloop.c`, `src/funcprog.c`, `src/ndarray.c`, `src/pack.c`, `src/part.c` · Result: the return-series kernel 2.02 s → 55.6 ms, image pipeline 2.19 s → 103 ms; both now at or ahead of NumPy

Common method in [README.md](#).

Hypothesis

The third sweep ([HPC_SWEEP_2_APPLICATIONS.md](#)) put Mathilda ahead of Mathematica on the array-heavy application kernels and left two rows badly behind NumPy: the return series at 66×, the image pipeline at 19×.

Those are not linear-algebra rows, so BLAS is not the answer. The hypothesis was that the remaining cost was in primitives, not compositions — and specifically in two categories the previous sweeps had not touched: structural operations (an $O(1)$ or memcpy operation walking a boxed list) and scans.

So this sweep started by probing primitives directly against their NumPy equivalent, rather than by running applications.

The probe (10^6 float64)

op	Mathilda	NumPy	ratio
Most[v]	270 ms	0.48 ms	563×
Rest[v] / v[1:]	223 ms	0.48 ms	466×
FoldList[a #1+b #2&,...] packed / lfilter	1246 ms	3.02 ms	413×
FoldList[Max,...] / maximum.accumulate	605 ms	2.73 ms	222×
ListConvolve[k, v] / np.convolve	263 ms	1.40 ms	188×

op	Mathilda	NumPy	ratio
First[v] / v[0]	123 ms	~0	O(1) done in O(n)
Clip[v, {a,b}] / np.clip	356 ms	4.66 ms	76×
Differences[v] / np.diff	6.6 ms	0.41 ms	16×
ListCorrelate[k5×5, im] / correlate2d	376 ms	52.5 ms	7.2×
Accumulate[v] / np.cumsum	7.9 ms	2.09 ms	3.8×
v v	0.81 ms	0.45 ms	1.8× ✓
Log[v]	8.8 ms	6.8 ms	1.3× ✓

The bottom two rows are the control: the elementwise kernels were already at the memory floor, so nothing above them is about arithmetic. Everything expensive was structural or a scan.

First[v] at 123 ms is the one that matters most. That is not a constant factor — it is an O(1) element read costing O(n), next to Drop[v, 250] on the same data at 0.88 ms.

Six fixes

1. **First / Last / Most / Rest** read the buffer

All four are leading-axis slices — one row, or all-but-one — so each is a pointer read or a single memcpy. All four went through ndstruct_delist_repack instead, which materialises every element to build a List and then repacks the answer.

	before	after
First[v]	123 ms	0.01 ms
Last[v]	103 ms	0.00 ms
Most[v]	270 ms	3.42 ms
Rest[v]	223 ms	0.71 ms

A rank-1 First/Last yields a scalar, so it routes through ndarray_element_to_expr — the single place that decides an element's head — and an int64 buffer still yields an exact Integer. Rank ≥ 2 drops the leading axis (First of a 3×4 is a length-4 vector, not a 1×4 matrix). Rest/Most of a single row is {}, which no buffer shape holds, so it degrades.

2. **Clip** back on the buffer, gated on the bounds

Clip had a complete buffer implementation and had been removed from the aware list entirely, because an exact bound makes the answer non-uniform:

```
Clip[{-2., 0., 2.}, {-1, 1}] -> {-1, 0., 1}      (mixed; unpackable)
Clip[{-2., 0., 2.}, {-1., 1.}] -> {-1., 0., 1.}    (uniform float64)
```

The exactness problem was real but belonged to the bounds, not to the head. Real bounds stay on the buffer; an exact bound is taken only when an in-range scan proves nothing is clipped, where the answer just is the input. Everything else degrades. 356 ms → 0.93 ms.

3 & 4. Scans

Two orthogonal problems. numloop_fold_impl — the compiled-VM path — bailed outright on a packed argument, so Fold over a buffer was not merely unaccelerated, it was slower than over the equivalent plain list, because the interpreter then materialised the whole array to walk it. And the VM has no OP_MAX, so a running maximum could not compile at all.

- `ndred_scan` — a two-line C loop for the four associative operators (Plus, Times, Max, Min), recognising both the bare-symbol spelling (Max) and the pure-function one (Max[#1, #2] &), which used to differ by 130 ms.
- `numloop_fold_impl` now reads a rank-1 real buffer directly and emits one.

	before	NumPy	after
<code>FoldList[Max, ...]</code> (running max)	605 ms	2.73 ms	2.00 ms
<code>FoldList[Max[#1,#2] &, ...]</code>	497 ms		1.66 ms
<code>FoldList[Plus, ...]</code>		2.09 ms	1.35 ms
<code>EMA, FoldList[0.98 #1 + 0.02 #2 &, ...]</code>	1246 ms	3.02 ms (lfilter)	22.6 ms

The running maximum is now faster than `np.maximum.accumulate`. The general linear recurrence stays on the VM at 22 ns/element, which is $7.5\times$ lfilter — down from $413\times$.

The exactness gate is the interesting part: Max returns one of its arguments, so an exact seed beside a Real buffer (`Fold[Max, 1, {0.5}]` → the exact 1) escapes the buffer's dtype and must decline. Times over int64 overflows to a bigint and abandons the whole scan.

5. ListConvolve / ListCorrelate on the buffer

Both engines — the direct $O(L\cdot m)$ loop and the FFT — already worked on flat double arrays. What they lacked was a way to get one without boxing every pixel first. Four changes, each measured:

	5×5 over 1024^2	5-tap over 10^6
before	376 ms	263 ms
read/emit buffers instead of leaves	195 ms	45.8 ms
+ hoist the interior case out of the padding logic	72.6 ms	30.4 ms
+ affine-stride inner loop	79.5 ms	31.5 ms
+ odometer instead of index division	46.5 ms	16.1 ms
NumPy	52.5 ms (correlate2d)	1.40 ms (np.convolve)

Two of those deserve comment.

The interior hoist: once an output is known to touch no padding, the list index for kernel element k is `flat_base + koff[k]` with `koff` fixed for the whole sweep, so the innermost body collapses from a per-axis index rebuild plus a range test plus a padding branch down to one load and one multiply-add. The boundary outputs still take the general body, so no answer changes.

The odometer was the surprise. The output multi-index was computed as $(o / Lstr[ax]) \% Ldims[ax]$ — a 64-bit integer division per axis per output, ~ 20 – 40 cycles each, with runtime strides the compiler cannot strength-reduce. On a rank-1 filter that was the entire cost: 10^6 divisions swamping 5×10^6 multiply-adds, and the vectorisable interior loop never got to show. Note that the affine-stride step measured worse than the interior hoist alone until the divisions were removed — it was being hidden.

The rank-2 correlation is now faster than SciPy's `correlate2d`.

6. Everything the re-probe still showed

- **Accumulate** — the float64 arm went through `ndt_get/ndt_set` (two indirect calls per element), and at rank 1 the T-inner loop emitted a full loop prologue around a single add. 7.9 ms → 3.5 ms.
- **Differences** — same choke point. 6.6 ms → 0.97 ms.
- **Part** with a contiguous span — the general path costs a modulo and a division per axis per element plus an indirect position lookup, where a unit-step selector on every axis is a block copy. 14.0 ms → 7.4 ms.

- **Outer** with one operand below the packing threshold — the small-operand trap from the third sweep, again: Outer[Plus, v100000, v16] at 849 ms → 10.9 ms.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Return series (EMA/vol/drawdown), 10 ⁶	2.02 s	55.6 ms	154 ms	30.4 ms	2.78× faster than WL
Gaussian blur + Sobel, 1024 ²	2.19 s	103 ms	94.5 ms	114 ms	at parity, 1.10× ahead of
ListConvolve, 1024 ² , 5×5	312 ms	35.7 ms	33.8 ms	39.7 ms	1.11× ahead of NumPy
Differences, 10 ⁷	56.6 ms	16.3 ms	14.3 ms	15.6 ms	
Accumulate, 10 ⁷	63.8 ms	19.4 ms	15.1 ms	28.7 ms	1.48× ahead of NumPy
k-means, 100000×8, k=16	2.26 s	2.24 s	7.10 s	767 ms	3.17× faster than WL
N-body, 1024 bodies	439 ms	383 ms	7.04 s	400 ms	18.4× faster than WL

The two rows this sweep targeted moved 36× and 21×, and both crossed from "worst in the suite" to at-or-ahead of NumPy.

A separate finding: **dgemm** slows the next threaded loop by 1.45×

The Jacobi row read 223 ms in the full run and 128 ms measured alone, while every other row was stable between the two. Bisecting the benchmark prefix found a single trigger — running one matrix multiply first:

Jacobi, 512 ² , 100 sweeps	
alone	130.3, 132.2, 132.9 ms
after matmul (1000×1000 dgemm)	188.6, 190.2, 190.5 ms

Reproducible to ±1%. Accelerate's threaded dgemm leaves the process in a state where Mathilda's own nd_parallel_for runs 1.45× slower — most likely its worker threads persisting and competing for cores.

This is not a regression from this sweep, and it is not confined to the benchmark: any program that does a matrix multiply and then array work pays it. It is recorded as an open item rather than fixed, because the fix is a thread-pool policy question (yield, pin, or share Accelerate's pool) that wants its own experiment. The Jacobi and Game of Life rows in the full-run table should be read with this in mind — they are the two threaded-stencil rows and they follow the linear-algebra group.

What the tests missed — three times

This is the most transferable result of the sweep.

1. A test asserted the fallback, not the invariant. `chk_eq("NDArrayQ[Clip[NDArray[{-2., 0., 2.}]]]", "True")` asserted that `Clip` returned a buffer for the 1-argument form — i.e. that it answered `{-1., 0., 1.}` where the `List` path answers `{-1, 0., 1}`. It was enshrining a wrong answer, and the two `chk_array` checks beside it could not catch it because they compare numeric distance and the difference is in element heads.
2. A regression was caught only by comparing two spellings. Teaching `numloop` to read a buffer made `FoldList[Plus, 0., NDArray[...]]` answer with head `NDArray` and `FoldList[Function[{p,q}, p+q], 0., NDArray[...]]` answer with head `List` — the same operation, two presentations. No single-path test would have seen it; the cross-spelling parity test in `test_compile.c` did.
3. Tests below the packing threshold test nothing. (Second appearance; see experiment 10.) Every new differential case in this sweep uses data above `PACK_MIN_ELEMENTS` for exactly this reason.

`test_fourth_sweep_fast_paths` in `tests/test_packed_list.c` adds ~200 differential cases built on those three rules: every path packed-vs-plain, every form each path must decline, and explicit presentation-parity

assertions.

Verification

- Full 395-binary suite: clean apart from one pre-existing `simplify_tests` failure (a symbolic radical case), confirmed pre-existing in the previous sweep.
- `make check-c99` and `make check-packed-aware` pass.
- Every benchmark value agrees across all three systems.

Still open

- **logreg** at 3.06× NumPy is now the largest application gap, and its cause is precise: `Transpose[X]` on a 200000×32 matrix costs 27 ms and the kernel does it 100 times, because it is loop-invariant but re-evaluated. NumPy's `.T` is a view and costs nothing. Closing this needs a strided NDArray view, which every consumer would have to honour — a real design change, not a fast path.
- k-means at 2.92× NumPy is unfused array composition: three passes over 6.4 MB where one would do. Interpreter-level fusion, not just inside `Compile[]`, is the fix.
- The 1-D convolution is 11× NumPy: the per-output interior test is now the cost, and splitting the output range into boundary/interior/boundary would remove it.
- `Part` with a span still materialises a position array per axis; a `start/step/n` selector representation would make it a pure memcpy.
- `ArrayReshape` does not exist as a builtin at all.

Why Mathilda is not the fastest here, and what it would take

This sweep took the two worst application rows from 66.5× and 19× behind NumPy to ahead of it, and put the running maximum ahead of `np.maximum.accumulate`. What the closing probe still showed:

row	Mathilda	best other	gap	cause
1-D ListConvolve, 5-tap over 10 ⁶	16.1 ms	1.40 ms (NumPy)	11.5×	a per-output boundary test in the inner loop
General linear recurrence (EMA)	22.6 ms	3.02 ms (<code>lfilter</code>)	7.5×	the compiled VM at 22 ns/element
Tally, 10 ⁷	—	—	7.2×	hashing, not boxing
Part with a span	7.4 ms	—	—	still builds a position array per axis
Reverse, RotateLeft	—	—	2.9–3.8×	serial

The road to fastest

1. Split the convolution output range into boundary / interior / boundary. The interior body needs no range test at all — the fourth sweep's odometer change removed the index division and left the test, which is now the whole cost. Expected: 16.1 ms → ~3 ms, close to NumPy.
2. A **start/step/n** selector for spans (plan 9.5), which makes `v[[a ;; b]]` a pure memcpy and also removes the position array from experiment 12's gather and experiment 5's strided sieve write. One change, three experiments.
3. Thread the block moves — `Reverse`, `RotateLeft` — see [06-packed-arrays/](#).
4. The linear recurrence is genuinely sequential and 22 ns/element is the VM's per-element dispatch. Closing this needs the codegen work in [03-compile-optimising-codegen/](#)'s roadmap rather than anything about scans.

Item 2 is the highest-leverage line in this list because three separate experiments are waiting on it.